

Ethical Hacking Assessment Report

Penetration Test of a Deliberately Vulnerable Web
Application — VulnLab

TESTER	Felix Otieno
GITHUB	github.com/felixotieno1011-art
DATE	30 September 2026
TARGET	http://127.0.0.1:5000 (localhost, own device)
APPLICATION	VulnLab — a deliberately vulnerable Flask application
SCOPE	Full application, all 16 endpoints
METHODOLOGY	Manual testing (browser, curl, Python)
PURPOSE	Educational penetration test on a self-built lab

9

CRITICAL

5

HIGH

2

MEDIUM

16

TOTAL FINDINGS

Table of Contents

1. Executive Summary

2. Scope and Objectives

3. Methodology

4. Summary of Findings

5. Detailed Findings

● 5.2 SQL Injection in Login (Authentication Bypass)	Critical
● 5.5 Broken Access Control in /admin (Cookie Forgery)	Critical
● 5.6 OS Command Injection in /ping (RCE)	Critical
● 5.9 Server-Side Template Injection (SSTI → RCE)	Critical
● 5.12 Insecure Deserialization (pickle → RCE)	Critical
● 5.14 Weak / Hardcoded Session Secret Key	Critical
● 5.16 Werkzeug Interactive Debugger Exposed	Critical
● 5.1 Reflected XSS in Search	Critical
● 5.8 Unrestricted File Upload → Stored XSS	Critical
● 5.4 IDOR in Profile and Orders	High
● 5.11 SSRF with LFI via /fetch	High
● 5.13 XXE Injection in XML Parser	High
● 5.3 Stored XSS in Guestbook	High
● 5.7 Path Traversal / LFI in File Viewer	High
● 5.15 CSRF in Password Change	Medium
● 5.10 Open Redirect in /redirect	Medium

6. Consolidated Recommendations

7. Ethical and Legal Considerations

8. Conclusion

1. Executive Summary

As part of this project, a structured, authorised penetration test was carried out against VulnLab, a deliberately vulnerable Flask web application built for hands-on ethical hacking practice. The objective was to identify, safely exploit, and document security weaknesses using a manual, methodology-driven approach consistent with industry practice (OWASP Testing Guide / OWASP Top 10), and to propose concrete remediation for each issue found.

The assessment identified **16 distinct vulnerabilities** spanning input validation, authentication, session management, access control, and server configuration. Of these, **9 were rated Critical, 5 were rated High, and 2 were rated Medium** in severity. Several findings — most notably the SQL injection in the login form, the command injection in the ping utility, the server-side template injection in the greeting page, the insecure pickle deserialization, and the exposed Werkzeug debugger — independently provide a path to complete remote code execution and therefore represent a total compromise of the application and its host.

A recurring theme across the findings is the direct use of untrusted user input to build executable artefacts — SQL strings, HTML, shell commands, Jinja templates, file paths, and pickled objects — instead of treating that input strictly as data. A second recurring theme is the application's implicit trust in client-supplied values (session role claims, URL identifiers, redirect targets) without server-side verification. Both patterns map directly onto well-established weakness categories in the OWASP Top 10 (2021), and the recommended fixes in Section 6 address them at the root cause rather than on a case-by-case basis.

2. Scope and Objectives

The scope of this assessment was limited strictly to VulnLab, running on a local, isolated host at `127.0.0.1:5000`, covering all 16 application endpoints. No external systems, production services, or third-party infrastructure were in scope or were tested.

- Identify vulnerabilities present in the application across its web endpoints, authentication flow, session handling, and file-handling functionality.
- Demonstrate the practical impact of each vulnerability through a safe, controlled proof of concept.
- Map each finding to the relevant CWE category and estimate its severity using CVSS-style scoring.
- Provide actionable, root-cause remediation guidance for each finding, together with a method to verify the fix.
- Document the assessment in a format suitable for a personal ethical-hacking portfolio.

3. Methodology

Testing followed a standard manual web-application penetration testing workflow, broadly aligned with the OWASP Testing Guide:

1. **Reconnaissance** — mapping the application's routes, forms, and parameters by manual browsing and reviewing available source/configuration where accessible.
2. **Vulnerability identification** — probing each input point (query parameters, form fields, cookies, headers, and uploaded files) for signs of unsafe handling.
3. **Exploitation** — constructing minimal proof-of-concept payloads to confirm each suspected vulnerability was genuinely exploitable, not merely theoretical.
4. **Impact analysis** — assessing what an attacker could realistically achieve through each finding, including chaining multiple findings together where relevant.
5. **Root-cause analysis** — reviewing the underlying application code to identify precisely why each vulnerability exists.
6. **Remediation and verification** — proposing a concrete fix for each issue and describing how to confirm the fix is effective.

All testing was performed using standard, freely available tools: a web browser, curl, and short Python scripts for constructing payloads (e.g. signed cookies, pickle payloads). No automated scanning or exploitation frameworks were required, since the application's weaknesses were directly reachable through its normal, documented interface.

4. Summary of Findings

The table below lists every finding in descending order of severity, with its CVSS score, location, and status. Full technical detail for each is provided in Section 5.

#	VULNERABILITY	SEVERITY	CVSS	LOCATION
2	SQL Injection in Login (Auth Bypass)	CRITICAL	9.8	/login
5	Broken Access Control in /admin (Cookie Forgery)	CRITICAL	9.8	/admin
6	OS Command Injection in /ping (RCE)	CRITICAL	9.8	/ping
9	Server-Side Template Injection → RCE	CRITICAL	9.8	/greet?name=
12	Insecure Deserialization (pickle) → RCE	CRITICAL	9.8	/cookie ('prefs' cookie)
14	Weak / Hardcoded Session Secret Key	CRITICAL	9.8	app.py – secret_key
16	Werkzeug Interactive Debugger Exposed	CRITICAL	9.8	Any unhandled exception
1	Reflected XSS in Search	CRITICAL	9.0	/search?q=
8	Unrestricted File Upload → Stored XSS	CRITICAL	9.0	/upload → /uploads/<name>
4	IDOR in Profile and Orders	HIGH	8.6	/profile?id=, /orders?id=
11	SSRF with LFI via /fetch	HIGH	8.6	/fetch?url=
13	XXE Injection in XML Parser	HIGH	8.2	/xml
3	Stored XSS in Guestbook	HIGH	8.0	/guestbook
7	Path Traversal / LFI in File Viewer	HIGH	7.5	/files?name=
15	CSRF in Password Change	MEDIUM	6.5	/password
10	Open Redirect in /redirect	MEDIUM	6.1	/redirect?next=

Severity distribution across all 16 findings: **9 Critical, 5 High, 2 Medium.**

5. Detailed Findings

Each finding below is presented with its description, reproduction steps, proof of concept, business impact, root cause in the application's source, recommended fix, and a method to verify that the fix closes the issue.

Finding 1 — Reflected Cross-Site Scripting (XSS) in Search

CRITICAL

CVSS: 9.0 Location: /search?q= CWE: CWE-79 Status: Confirmed and exploited

DESCRIPTION

The search page reflects the value of the 'q' query parameter directly into the HTML response without any output encoding. Any JavaScript submitted through this parameter executes in the victim's browser in the security context of the application. Although the Flask session cookie is marked HttpOnly (so `document.cookie` cannot read it directly), the injected script still runs with the victim's authenticated session, enabling actions to be performed on the victim's behalf — a technique often called session riding.

STEPS TO REPRODUCE

1. Log in to the application as any user (e.g. felix / felix123) at /login.
2. Navigate to the search page at /search.
3. Enter the payload `<script>alert("XSS")</script>` into the search box and submit.
4. Observe that a JavaScript alert box fires, confirming the input was executed as code rather than displayed as text.

PROOF OF CONCEPT

Payload 1 — confirm execution

```
<script>alert("XSS")</script>
```

A popup showing "XSS" appears, proving arbitrary JavaScript executes in the page context.

Payload 2 — read a protected page as the victim

```
<script>fetch('/admin').then(r=>r.text()).then(t=>alert(t.slice(0,500)))</script>
```

The popup displays HTML from the /admin panel, proving the victim's own session was used to fetch a page the attacker could not otherwise access.

Payload 3 — silent account takeover

```
<script>fetch('/password',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:'new=HACKED123'})</script>
```

The victim's password is silently changed to 'HACKED123' with no visible warning. It was verified that logging in afterwards with the new password succeeded, confirming full account takeover.

IMPACT

- Arbitrary JavaScript execution in a victim's browser, delivered via a single crafted link.
- Theft or abuse of the victim's authenticated session to perform any action the site permits.
- Silent, unauthorised password changes leading to full account takeover.
- Potential for phishing redirects, page defacement, or keystroke logging if extended.

ROOT CAUSE

app.py builds the response with an f-string: `body = f'...You searched for: {q}...'.` The value of `q` is taken from `request.args` and interpolated directly into raw HTML. Because this bypasses Jinja's automatic escaping (which only applies to `{{ }}` template expressions, not to values embedded in an f-string before rendering), any HTML or script tags in the input are treated as executable markup by the browser.

RECOMMENDED FIX

- ✓ Escape user input before it is placed into HTML: `from markupsafe import escape; safe_q = escape(q).`
- ✓ Preferably avoid f-string HTML construction entirely and pass the value through Jinja templating, e.g. `render_template_string("<p>You searched for: {{ q }}</p>", q=q),` which auto-escapes by default.
- ✓ Add a Content-Security-Policy header (default-src 'self'; script-src 'self') as a defence-in-depth control against injected inline scripts.

- ✓ Where rich HTML input is genuinely required, sanitise it through an allow-list library such as bleach.

Verification after fix: Submitting `<script>alert("XSS")</script>` should cause the literal text of the tag to be displayed on the page rather than executed, and no popup should appear.

Finding 2 — SQL Injection in Login (Authentication Bypass)

CRITICAL

CVSS: 9.8 Location: /login CWE: CWE-89 Status: Confirmed and fully exploited

DESCRIPTION

The login form builds its SQL query by directly concatenating the submitted username and password into a query string. Any SQL metacharacter supplied by the user is interpreted as part of the query itself, allowing authentication bypass, arbitrary data disclosure, and — with more advanced payloads — data modification.

STEPS TO REPRODUCE

1. Navigate to /login.
2. Enter one of the payloads below in the Username field (password can be anything).
3. Click 'Sign in'.

PROOF OF CONCEPT

Payload 1 — bypass as a named user

```
Username: admin' -- | Password: anything
```

Login succeeds as admin. The single quote closes the string literal and '--' comments out the remainder of the query, so the password check is never evaluated.

Payload 2 — bypass as any other user

```
Username: mary' -- | Password: anything
```

The same technique authenticates as any username that exists in the users table.

Payload 3 — login without knowing any username

```
Username: ' OR 1=1 -- | Password: x
```

The always-true condition (1=1) causes the query to match the first row in the table, logging the attacker in as admin.

Payload 4 — full data exfiltration via UNION

```
Username: ' UNION SELECT 1,username,password,role,email FROM users -- | Password: x
```

The injected UNION query returns every user's username, password, role and email in a single request, in addition to authenticating as admin.

IMPACT

- Complete authentication bypass — an attacker with no valid credentials can log in as any user, including admin.
- Disclosure of every user's username, password, role and email in plaintext.
- With extended payloads, potential to modify or delete data in the underlying database.
- Compromises the confidentiality and integrity of the entire application.

ROOT CAUSE

app.py builds the query as: `f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"`. Because the values are concatenated as raw strings rather than passed as bound parameters, the database cannot distinguish user-supplied data from SQL syntax.

RECOMMENDED FIX

- ✓ Use parameterised queries (prepared statements) throughout: `conn.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))`. The sqlite3 driver treats the placeholders as data, never as executable SQL.
- ✓ Hash passwords with a strong algorithm (bcrypt or argon2) rather than storing them in plaintext.
- ✓ Return a generic 'Invalid credentials' message on failure — never surface database errors to the user.

- ✓ Log and monitor repeated failed login attempts to detect brute-force or injection probing.

Verification after fix: With parameterised queries in place, the payload "admin' --" is treated as the literal username "admin' --", which does not exist in the database, so the login attempt correctly fails.

Finding 3 — Stored Cross-Site Scripting (XSS) in Guestbook

HIGH

CVSS: 8.0 Location: /guestbook CWE: CWE-79 Status: Confirmed and exploited

DESCRIPTION

The guestbook feature stores visitor messages in the database and renders them back without HTML escaping. Because the payload persists server-side, it executes automatically for every subsequent visitor who opens the guestbook page — no phishing link or additional user action is required, which makes this more dangerous than the reflected XSS in Finding 1.

STEPS TO REPRODUCE

1. Navigate to /guestbook.
2. Submit a guestbook entry containing: `<img src=x onerror="alert('Stored XSS Works')">`.
3. Click Post, then refresh the page.
4. Observe that a JavaScript alert fires automatically, confirming the payload is stored and executes on every load.

PROOF OF CONCEPT

Bypass of naive <script> filtering

```
<img src=x onerror="alert('Stored XSS Works')">
```

Using a broken image source to trigger the onerror handler bypasses simple <script>-tag filtering that some browsers or naive filters apply. The alert fires for every visitor, logged in or not, across sessions and application restarts, until the database entry is removed.

IMPACT

- Persistent JavaScript execution for every visitor to the guestbook page, requiring no user interaction beyond loading it.
- Potential theft of non-HttpOnly cookies, or abuse of any logged-in visitor's session, including an administrator's.
- Ability to redirect all visitors to a phishing site or deface the page for every viewer.
- Could be used as a self-propagating worm by posting further malicious guestbook entries.

ROOT CAUSE

app.py interpolates the stored author and message fields directly into an HTML string with an f-string (`f"<p>{r['message']}</p>"`) rather than through an auto-escaping template mechanism, so any HTML or JavaScript in a stored message is rendered as executable markup.

RECOMMENDED FIX

- ✓ Escape stored content before rendering: `from markupsafe import escape; escape(r['message'])`.
- ✓ Prefer rendering through Jinja templates (`{{ r.message }}`), which auto-escape by default — never apply the `|safe` filter to user-controlled fields.
- ✓ Add a Content-Security-Policy header to block inline script execution even if an injection occurs.
- ✓ Sanitise input at submission time as a second, defence-in-depth layer, using an allow-list library such as bleach.

Verification after fix: Posting `<img src=x onerror="alert('XSS')">` should display as literal text in the guestbook with no image element rendered and no popup triggered.

Finding 4 — Insecure Direct Object Reference (IDOR) in Profile and Orders

HIGH

CVSS: 8.6 Location: /profile?id=, /orders?id= CWE: CWE-639 Status: Confirmed and fully exploited

DESCRIPTION

Both the /profile and /orders endpoints accept an 'id' parameter from the URL and return the corresponding record with no check that the requester owns that record, and no requirement to be logged in at all. Any visitor can enumerate every record simply by incrementing the id. The profile endpoint additionally returns the user's password in plaintext, compounding the impact.

STEPS TO REPRODUCE

1. Navigate to /profile?id=1 — the full record for the admin account is returned, including the password field.
2. Change the id parameter (?id=2, ?id=3, ?id=4) to view other users' profiles without authentication.
3. Log out entirely and request /profile?id=1 again — the record is still returned, showing no auth check exists.
4. Repeat against /orders?id=1 to retrieve another user's order, including a sensitive note (a lab flag was captured in this field).

PROOF OF CONCEPT

Enumeration across both endpoints

```
/profile?id=1..4 and /orders?id=1
```

All four user profiles, including plaintext passwords, and the admin's order record (containing a marked flag string) were retrieved with no authentication.

IMPACT

- Full disclosure of every user's credentials with no need to crack password hashes, since they are stored in plaintext.
- Any attacker can subsequently log in as any disclosed user.
- Disclosure of sensitive order data belonging to other accounts.
- Combined with the SQL injection in Finding 2, the database is effectively fully exposed by two independent routes.

ROOT CAUSE

app.py builds queries such as `f"SELECT * FROM users WHERE id = {uid}"` directly from the URL parameter, with no authentication decorator and no ownership check — and, as an f-string, this endpoint is also independently vulnerable to SQL injection.

RECOMMENDED FIX

- ✓ Require authentication on both endpoints using a login_required decorator.
- ✓ Never trust an id supplied in the URL for the current user's own data — derive it from the server-side session instead: `uid = session["user_id"]`.
- ✓ Where an id must be accepted (e.g. an admin viewing another user's record), explicitly verify authorisation: `if uid != session["user_id"] and session.get("role") != "admin": abort(403)`.
- ✓ Remove the password field from any API or page response entirely — it should never leave the server.
- ✓ Use parameterised queries to close the accompanying SQL injection vector.

Verification after fix: Unauthenticated requests are redirected to /login, a logged-in user requesting another user's id receives HTTP 403, and no response body contains a password field.

Finding 5 — Broken Access Control in /admin via Session Cookie Forgery

CRITICAL

CVSS: 9.8 Location: /admin CWE: CWE-287, CWE-565 Status: Confirmed and fully exploited

DESCRIPTION

The /admin endpoint authorises access purely by reading the 'role' field from the Flask session cookie. Flask signs this cookie using the application's secret key; because the key used here is a known, hardcoded string (see Finding 14), an attacker who knows or obtains it can forge a validly signed cookie declaring role="admin" and gain full administrative access without ever logging in. A second, independent bug compounds this: the endpoint trusts the role claimed inside the cookie rather than re-checking it against the database.

STEPS TO REPRODUCE

1. Read app.py (or obtain it via Finding 7's path traversal) to find the hardcoded secret key.
2. Using Flask's own signing serializer in Python, construct a session payload of {"user_id": 2, "role": "admin"} and sign it with the known key.
3. Send the forged cookie to the server: `curl -b "session=<forged>" http://127.0.0.1:5000/admin`.
4. Observe that the admin panel is returned in full, listing all users, with no login having taken place.

PROOF OF CONCEPT

Forged admin cookie

```
eyJ1c2VyX2lkIjoyLCJyb2x1IjoieWRtaW4ifQ.aryEMA.WpeKEUuP6HiiKoIzV4zDxcj5PTk
```

Sending this cookie to /admin returned the full admin panel HTML, including every user's role and email. It was also noted that the forged cookie declares user_id=2 (a normal user) with role="admin" — the server only checks the claimed role field and never re-verifies it against user 2's actual role in the database.

IMPACT

- Complete, unauthenticated access to the administrative panel and all data it exposes.
- Combined with the IDOR in Finding 4, every user's plaintext password becomes reachable through the same trust failure.
- Combined with the command injection in Finding 6, an attacker escalates from data exposure to full server compromise.

ROOT CAUSE

Two compounding issues in app.py: (1) `app.secret_key = "insecure-secret-key-for-learning"` is hardcoded and guessable/known; (2) the admin view checks `role = session.get("role")` directly rather than looking the user's role up from the database.

RECOMMENDED FIX

- ✓ Generate a strong, random secret key at deploy time and load it from an environment variable, never from source:
`app.secret_key = secrets.token_hex(32)`.
- ✓ Never trust a role claimed by the client. Always re-derive it from the database on every privileged request, e.g. via an `admin_required` decorator that loads the current user and checks `u["role"] == "admin"`.
- ✓ Set `SESSION_COOKIE_HTTPONLY`, `SESSION_COOKIE_SECURE` and `SESSION_COOKIE_SAMESITE` to harden the cookie itself.
- ✓ Rotate the secret key immediately if it is ever suspected to have been exposed.

Verification after fix: Previously forged cookies fail signature verification and are rejected. Even a correctly signed cookie claiming role="admin" is denied if the corresponding user's database record does not actually have that role.

Finding 6 — OS Command Injection in /ping (Remote Code Execution)

CRITICAL

CVSS: 9.8 Location: /ping CWE: CWE-78 Status: Confirmed and fully exploited

DESCRIPTION

The /ping endpoint takes a 'host' parameter and passes it directly into a shell command via `subprocess.check_output` with `shell=True`. Any shell metacharacter in the input (`;`, `&&`, `|`, backtick, `$()`) causes a second, attacker-controlled command to run with the privileges of the web service process, which in the tested environment ran as the Termux user `u0_a333` with full access to its home directory.

STEPS TO REPRODUCE

1. Navigate to /ping.
2. Submit the host value `127.0.0.1; whoami` and observe that the ping output is followed by the service account's username.
3. Escalate to further reconnaissance and file access using the same injection point.

PROOF OF CONCEPT

Identify the service account

```
127.0.0.1; whoami
```

Returned `u0_a333`, confirming command execution.

List the application directory

```
127.0.0.1; ls -la ~/learning/vuln_app
```

Returned the full directory listing, including `app.py`, `static/`, `templates/`, `uploads/` and the SQLite database file.

Read the hardcoded secret key

```
127.0.0.1; grep secret_key ~/learning/vuln_app/app.py
```

Returned the line `app.secret_key = "insecure-secret-key-for-learning"`, directly enabling the session forgery in Finding 5.

Prove file write capability

```
127.0.0.1; echo "I was here" > ~/learning/vuln_app/pwned.txt
```

Confirmed the attacker can create and modify files on the server, not just read them.

Dump credentials directly from the database

```
127.0.0.1; sqlite3 ~/learning/vuln_app/vuln_app.db "SELECT username,password FROM users;"
```

Returned every user's username and plaintext password.

IMPACT

- Full remote code execution with the privileges of the web application process.
- Ability to read, write or delete any file the service account can access, including source code, secrets and the user database.
- A practical path to a fully interactive reverse shell and to pivoting against other services on the same host.
- Rated as the single highest-impact class of vulnerability identified in this assessment.

ROOT CAUSE

`app.py` runs `subprocess.check_output(f"ping -c 1 {host}", shell=True, ...)`. Using `shell=True` causes the string to be interpreted by a shell, and the host value is concatenated into that string without any validation.

RECOMMENDED FIX

- ✓ Never use `shell=True` with user-controlled input. Pass arguments as a list with `shell=False`:
`subprocess.check_output(["ping", "-c", "1", host], shell=False, timeout=5).`
- ✓ Strictly validate the host value against an allow-list pattern before use, e.g. `re.fullmatch(r"[A-Za-z0-9.\-]{1,255}", host)`, rejecting anything else with HTTP 400.
- ✓ Consider whether exposing a raw ping utility through the web is necessary at all — the risk rarely justifies the feature.
- ✓ Run the service under a low-privilege account with no access to sensitive files, and consider sandboxing (containers, firejail) as defence in depth.

Verification after fix: `127.0.0.1; whoami` no longer executes the injected command, malformed hosts are rejected with HTTP 400, and a legitimate ping to a real address such as `8.8.8.8` continues to work.

Finding 7 — Path Traversal / Local File Inclusion in File Viewer

HIGH

CVSS: 7.5 **Location:** /files?name= **CWE:** CWE-22 **Status:** Confirmed and fully exploited

DESCRIPTION

The /files endpoint reads a user-supplied filename with no sanitisation. Supplying a relative path containing '..' sequences allows an attacker to escape the intended files/ directory and read any file the service process can access — in this environment, the whole home directory of the Termux user running the app.

STEPS TO REPRODUCE

1. Confirm normal operation: /files?name=welcome.txt returns the intended file.
2. Supply a traversal payload: /files?name=../app.py — the entire application source code is returned.
3. Read the database file directly: /files?name=../vuln_app.db — the raw SQLite file is returned, exposing plaintext user records.

PROOF OF CONCEPT

Source code disclosure

```
/files?name=../app.py
```

Returned the full app.py source, including the hardcoded secret key.

Database disclosure

```
/files?name=../vuln_app.db
```

Returned the raw SQLite database; readable strings inside it included user email addresses and plaintext passwords.

Attempted escape to /etc/passwd

```
/files?name=../../../../../../../../etc/passwd
```

Failed in this Android/Termux environment because that path does not exist under Termux's non-standard filesystem layout — the traversal logic itself succeeded, and the same payload would return /etc/passwd on a standard Linux deployment.

Confirm arbitrary read within the home directory

```
/files?name=../../.bashrc
```

Returned the full contents of the user's shell configuration file.

IMPACT

- Disclosure of the complete application source code.
- Disclosure of the Flask secret key, directly enabling the session-forgery attack in Finding 5.
- Disclosure of the SQLite database, including usernames, emails and plaintext passwords.
- On a standard server layout, potential exposure of SSH keys, cloud credentials and other sensitive configuration outside the application directory.

ROOT CAUSE

app.py builds the target path with `os.path.join(BASE_DIR, "files", name)`, but `os.path.join` does not strip or reject '..' sequences, so a crafted name escapes the intended files/ directory.

RECOMMENDED FIX

- ✓ Resolve the requested path with `os.path.realpath` and verify the result still lies inside the allowed directory before opening it: `if not target.startswith(ALLOWED_DIR + os.sep): return "Access denied", 403.`
- ✓ Reject any filename containing '..' or a leading '/' as an early, explicit check.
- ✓ Strip any directory components from the filename with `os.path.basename` before joining it to the allowed directory.

- ✓ Run the service under an account with read access limited to the files/ folder alone, not the wider project or home directory.

Verification after fix: /files?name=../app.py and /files?name=../.bashrc return HTTP 403, while /files?name=welcome.txt continues to work as intended.

Finding 8 — Unrestricted File Upload → Stored XSS and Source Disclosure

CRITICAL

CVSS: 9.0 **Location:** /upload → /uploads/<name> **CWE:** CWE-434, CWE-79 **Status:** Confirmed and fully exploited

DESCRIPTION

The /upload endpoint accepts a file of any type, saves it into the uploads/ directory, and serves it back at /uploads/<name> with no content-type restriction, extension allow-list, filename sanitisation, or size limit. Because uploaded files are served from the same origin as the application, an attacker who uploads an HTML file can achieve stored cross-site scripting that affects every visitor to that URL. Uploading a source file such as a .py script also discloses its contents as plain text, and on a misconfigured production server that executes uploaded scripts, this same weakness would lead directly to remote code execution.

STEPS TO REPRODUCE

1. Confirm baseline behaviour: upload a plain text file via the form and retrieve it from /uploads/<name> to confirm the upload pathway works.
2. Upload an HTML file containing a script tag that pops an alert, then visit its /uploads/ URL directly.
3. Observe that the alert fires, confirming the file was served in a way that allows script execution within the application's own origin.
4. Separately, upload a Python file containing a system call and retrieve it from /uploads/ to confirm its full source is returned as plain text.

PROOF OF CONCEPT

Stored XSS via uploaded HTML

```
<h1>Upload XSS</h1><script>alert("XSS via uploaded HTML file!");</script>
uploaded as evil.html, then loaded at /uploads/evil.html
```

The browser executed the embedded script and displayed the alert, proving the file was served with a content type permitting script execution in the app's own origin — meaning any visitor who is tricked or linked into opening the uploaded file is affected exactly as with the guestbook stored XSS in Finding 3.

Source code disclosure via uploaded script

```
A Python file containing os.popen("id; whoami; pwd") uploaded as pwn.py,
then loaded at /uploads/pwn.py
```

The raw Python source was returned as plain text. The server did not execute the script in this configuration, but the same weakness would result in code execution wherever uploads are handled by a scripting engine (CGI, WSGI handler, PHP, JSP, etc.).

IMPACT

- Persistent, self-hosted cross-site scripting served from the application's own trusted origin, enabling session abuse, cookie theft of non-HttpOnly cookies, and phishing overlays for any visitor directed to the uploaded file.
- Disclosure of uploaded source code content, and a realistic path to full remote code execution on any deployment that processes uploads as executable scripts.
- No existence check on the destination filename, allowing existing uploaded files to be silently overwritten.
- The upload directory can be used as a staging area for phishing content hosted on a trusted domain, compounding Finding 10 (Open Redirect).

ROOT CAUSE

app.py saves the uploaded file directly using the client-supplied filename (`f.save(os.path.join(UPLOAD_DIR, f.filename))`) with no validation of extension, content type, or filename content, and serves it back with a plain `send_file` call and no restrictive headers.

RECOMMENDED FIX

- ✓ Enforce an extension allow-list (e.g. png, jpg, jpeg, gif, pdf) and reject anything else with HTTP 400.
- ✓ Sanitise the filename with Werkzeug's `secure_filename()` before using it to build a path.
- ✓ Store uploads outside the web-servable directory tree entirely, so they cannot be executed or interpreted by the

web server.

- ✓ Serve uploads through a dedicated handler that forces download rather than inline rendering, e.g. `send_file(path, as_attachment=True, download_name=name)`, and add `X-Content-Type-Options: nosniff` and a restrictive Content-Security-Policy to every response.
- ✓ Enforce a maximum upload size via `MAX_CONTENT_LENGTH`.
- ✓ Where possible, serve uploads from a separate origin/subdomain with no shared session or cookies, and re-encode image uploads server-side (e.g. with Pillow) to strip any embedded HTML/SVG content.

Verification after fix: Uploading `evil.html` or `pwn.py` is rejected with 'File type not allowed'; uploading a valid image succeeds and is served as an attachment with the `nosniff` header set; and no JavaScript executes from any file retrieved via `/uploads/`.

Finding 9 — Server-Side Template Injection (SSTI) leading to RCE

CRITICAL

CVSS: 9.8 Location: /greet?name= CWE: CWE-94 Status: Confirmed and fully exploited

DESCRIPTION

The /greet endpoint passes the 'name' parameter directly into render_template_string(). This means the entire input is interpreted as Jinja2 template syntax, not just displayed as data. By chaining Jinja expressions through Python's object model, an attacker can reach os.popen() and achieve complete remote code execution on the server.

STEPS TO REPRODUCE

1. Confirm baseline behaviour: /greet?name=Felix returns 'Hello, Felix!'.
2. Confirm template evaluation is occurring: /greet?name={{7*7}} returns 'Hello, 49!' rather than the literal expression.
3. Leak application configuration, including the secret key, via /greet?name={{config}}.
4. Enumerate loaded Python classes via the __mro__/__subclasses__ chain to locate a usable primitive.
5. Achieve full command execution via a Jinja global that exposes os, such as cycler.__init__.__globals__.os.popen('id').read().
6. Use the same technique to read the application's own source code.

PROOF OF CONCEPT

Confirm SSTI with arithmetic

```
{{7*7}}
```

Response reads 'Hello, 49!' — proof the expression was evaluated server-side rather than echoed as text.

Leak configuration and secret key

```
{{config}}
```

Response includes the full Flask config object, including SECRET_KEY = 'insecure-secret-key-for-learning'.

Enumerate classes for a usable gadget

```
{{'__.__class__.__mro__[1].__subclasses__()'}}
```

Returns a large list of loaded Python classes, including subprocess-related primitives.

Full remote code execution

```
{{cycler.__init__.__globals__.os.popen('id').read()}}
```

Returns the output of the `id` command, proving arbitrary OS command execution.

Read application source

```
{{cycler.__init__.__globals__.os.popen('cat app.py').read()}}
```

Returns the entire app.py source code.

IMPACT

- Full remote code execution within the web server process, equivalent in severity to Finding 6.
- Complete access to application source, the database, and any secrets on disk.
- A direct path to a reverse shell, data exfiltration, or a persistent backdoor.
- Combined with the forged sessions of Finding 5 and the known secret key of Finding 14, provides an independent, redundant path to total application compromise.

ROOT CAUSE

app.py builds the template itself from user input: `render_template_string(f"Hello, {name}!")`. Because `name` becomes part of the template source rather than a value passed into it, any template syntax it contains is compiled and executed.

RECOMMENDED FIX

- ✓ Never build template source strings from user input. Pass user data as a variable instead:
`render_template_string("Hello, {{ name }}!", name=name)` — here the input is treated purely as data, not code.
- ✓ Prefer `render_template()` with a static template file over `render_template_string()` wherever possible.
- ✓ If dynamic templates are unavoidable, use Jinja2's `SandboxedEnvironment` and strip dangerous globals.
- ✓ Validate the input against an allow-list of expected characters, e.g. `re.fullmatch(r"[A-Za-z0-9 _-]{1,64}", name)`.
- ✓ Run the service under a low-privilege account so that, even if SSTI recurs elsewhere, the resulting damage is limited.

Verification after fix: `/greet?name={{7*7}}` returns the literal text 'Hello, {{7*7}}!' rather than 'Hello, 49!', the config-leak and RCE payloads return literal text with no evaluation, and normal names continue to work correctly.

Finding 10 — Open Redirect in /redirect

MEDIUM

CVSS: 6.1 Location: /redirect?next= CWE: CWE-601 Status: Confirmed

DESCRIPTION

The /redirect endpoint takes a 'next' parameter and sends the user's browser there with no validation and no allow-list of permitted destinations. Because the resulting link begins with the trusted application's own hostname, it can be used to lend false credibility to phishing links, malware download links, or OAuth token-harvesting pages.

STEPS TO REPRODUCE

1. Confirm normal behaviour: /redirect?next=/ lands on the application's home page.
2. Supply an external destination: /redirect?next=https://example.com — the browser is redirected off-site with no warning.
3. Note that any URL is accepted, since the application performs no destination filtering.

PROOF OF CONCEPT

External redirect

```
/redirect?next=https://example.com
```

The browser navigated to example.com.

Simulated phishing link

```
/redirect?next=https://example.com/fake-login
```

The victim initially sees the trusted host (127.0.0.1:5000) in the address bar before landing on an attacker-controlled page, which is the exact property abused to make phishing links appear trustworthy or to slip past link-scanners that only check the leading hostname.

IMPACT

- Can be used to add legitimate-looking credibility to phishing campaigns.
- Can be chained with OAuth-style flows to intercept tokens.
- Can direct victims to malware downloads or bypass link scanners that only inspect the leading hostname.
- Generally rated Medium in isolation but escalates to High severity when chained into a larger attack.

ROOT CAUSE

app.py performs `return redirect(request.args.get("next", "/"))` with no validation of the supplied value whatsoever.

RECOMMENDED FIX

- ✓ Maintain an allow-list of permitted internal paths and/or hosts, and reject anything outside it with HTTP 400.
- ✓ Alternatively, restrict redirects to relative paths only, rejecting any value that does not start with a single '/' (guarding specifically against '/' which browsers may still treat as protocol-relative to an external host).
- ✓ Never reflect a raw destination into a Location header without validating both scheme and host.
- ✓ Consider an interstitial warning page for any redirect leaving the application's own domain.

Verification after fix: /redirect?next=/ and /redirect?next=/login continue to work, while /redirect?next=https://example.com, /redirect?next=//evil.com and /redirect?next=javascript:alert(1) are all rejected with HTTP 400.

Finding 11 — SSRF with Local File Inclusion via /fetch

HIGH

CVSS: 8.6 **Location:** /fetch?url= **CWE:** CWE-918, CWE-22 **Status:** Confirmed and fully exploited

DESCRIPTION

The /fetch endpoint takes a URL from the user and retrieves it with `urllib.request.urlopen()`, with no restriction on scheme or host. This allows an attacker to make the server issue requests to internal-only addresses on their behalf (SSRF) and, via the `file://` scheme, to read local files from the server's filesystem (LFI) — effectively using the trusted server as a network and filesystem proxy.

STEPS TO REPRODUCE

1. Confirm SSRF against the application's own home page: `/fetch?url=http://127.0.0.1:5000/`.
2. Confirm SSRF against a protected internal endpoint: `/fetch?url=http://127.0.0.1:5000/admin`.
3. Confirm `file://` scheme support by requesting the local database file directly.
4. Repeat against the application source file and the user's shell configuration file.

PROOF OF CONCEPT

SSRF against the app itself

```
/fetch?url=http://127.0.0.1:5000/
```

Returned the home page HTML, proving the server made the request on the attacker's behalf.

SSRF against a protected page

```
/fetch?url=http://127.0.0.1:5000/admin
```

Returned 'Forbidden — role required: admin', still confirming the server itself issued the internal request.

LFI via `file://` — database

```
/fetch?url=file:///.../vuln_app.db
```

Returned the raw SQLite file, containing user records and table definitions.

LFI via `file://` — source code

```
/fetch?url=file:///.../app.py
```

Returned the full application source.

LFI via `file://` — shell config

```
/fetch?url=file:///.../.bashrc
```

Returned the contents of the user's `.bashrc` file.

IMPACT

- Read access to any file reachable by the server process.
- Ability to enumerate and probe internal-only services from outside the network boundary.
- Bypasses network-level access controls by pivoting requests through the trusted server.
- On a cloud-hosted deployment, this class of bug commonly exposes the instance metadata service (169.254.169.254), which can lead to theft of cloud IAM credentials and full account takeover.

ROOT CAUSE

`app.py` performs `urllib.request.urlopen(url, timeout=5)` directly on the user-supplied URL, with no restriction on scheme (`http`, `https`, `file`, `ftp`, `gopher`, `dict` are all accepted) and no restriction on destination host, including loopback and private address ranges.

RECOMMENDED FIX

- ✓ Restrict the accepted URL scheme to http/https only, rejecting anything else with HTTP 400.
- ✓ Resolve the destination hostname and reject requests to private, loopback, or link-local address ranges using Python's `ipaddress` module.
- ✓ Where feasible, maintain an explicit allow-list of permitted destination hosts — the strongest available mitigation.
- ✓ Do not follow redirects to non-allow-listed hosts, and consider enforcing egress restrictions at the network layer (firewall rules or a network namespace) as defence in depth.

Verification after fix: `file://` URLs are rejected with 'Only http and https allowed', internal addresses such as 127.0.0.1 and 169.254.169.254 are rejected as internal, and legitimate external HTTP(S) URLs continue to work.

Finding 12 — Insecure Deserialization (pickle) leading to RCE

CRITICAL

CVSS: 9.8 Location: /cookie (via the 'prefs' cookie) CWE: CWE-502 Status: Confirmed and fully exploited

DESCRIPTION

The /cookie endpoint reads a base64-encoded value from the attacker-controlled 'prefs' cookie and passes it to `pickle.loads()`. Python's pickle format is not a passive data format — it is capable of instantiating arbitrary objects and invoking arbitrary callables during deserialization via the `__reduce__` protocol, so an attacker who controls the pickle stream controls what code runs on the server.

STEPS TO REPRODUCE

1. Open /cookie and note the default base64-encoded cookie value the application sets.
2. In Python, craft a class whose `__reduce__` method returns `(os.system, ("<command>",))` and serialise an instance of it with pickle, then base64-encode the result.
3. Send the crafted value as the 'prefs' cookie in a request to /cookie.
4. Confirm the command executed by inspecting its output or side effects on the server.

PROOF OF CONCEPT

Malicious pickle payload

```
A pickle built from a class whose __reduce__ returns
(os.system, ("id > ../pwned.txt",)), base64-encoded and sent as the 'prefs' cookie
```

The server's response 'Loaded prefs: 0' is the exit code of `os.system`, confirming the command ran successfully; the resulting file contained the output of the `id` command, proving remote code execution at the privilege level of the web process.

IMPACT

- Full remote code execution on the server, triggered by nothing more than a single crafted cookie header — no authentication or user interaction is required.
- Complete read/write/delete access to any file the process can reach.
- A direct path to a reverse shell or persistent backdoor (cron jobs, added SSH keys, web shells).
- Combined with any other finding in this report, guarantees total compromise of the host.

ROOT CAUSE

`app.py` executes `pickle.loads(base64.b64decode(request.cookies.get("prefs")))` on data that originates entirely from the client. The Python documentation explicitly states that pickle must never be used to deserialize untrusted data.

RECOMMENDED FIX

- ✓ Never unpickle data that originates from the client. This is the only fully correct remediation.
- ✓ Replace pickle with a safe, data-only serialisation format such as JSON: `json.loads(base64.b64decode(raw))`.
- ✓ If a legacy dependency on pickle truly cannot be removed, use a restricted unpickler that only permits an explicit allow-list of classes via a custom `find_class()` implementation.
- ✓ If cookie-based state is required, sign it with an HMAC and verify the signature before any decoding, or move the preference data to server-side storage entirely.

Verification after fix: The same crafted pickle payload is rejected or falls back to a safe default, no command executes and no file is created, while ordinary JSON-encoded preference cookies continue to work.

Finding 13 — XML External Entity (XXE) Injection in XML Parser

HIGH

CVSS: 8.2 Location: /xml CWE: CWE-611 Status: Confirmed — parser accepts external entity declarations

DESCRIPTION

The /xml endpoint parses user-submitted XML using Python's default SAX parser (`xml.sax.make_parser()`), which does not disable external entity resolution by default. An attacker can therefore submit a DOCTYPE declaration containing a SYSTEM entity that references a local file (`file://`) or an internal URL (`http://`), causing the parser to fetch and expand that external content — leaking arbitrary files on a standard server layout, and enabling SSRF via the `http://` scheme.

STEPS TO REPRODUCE

1. Send a POST request to /xml containing a DOCTYPE with a SYSTEM entity pointing at `file:///etc/passwd`.
2. Observe that the parser accepts the DOCTYPE without rejecting it.
3. Repeat with a SYSTEM entity pointing at an internal URL (`http://127.0.0.1:5000/admin`) to confirm the SSRF variant.

PROOF OF CONCEPT

XXE via internal URL (SSRF variant)

```
<?xml version="1.0"?><!DOCTYPE r [<!ENTITY x SYSTEM "http://127.0.0.1:5000/admin">]><r>START&x;END</r>
```

The parser accepted the DOCTYPE and attempted entity expansion (the response returned 'STARTEND', with the entity expanding to empty content in this specific test environment because Termux's Android sandbox restricted the underlying file/network read at parse time). This still confirms the vulnerability: a securely configured parser would have rejected the DOCTYPE outright with an 'external entity not allowed' error, whereas this parser silently accepted it. On a standard Linux server, the equivalent payload targeting `file:///etc/passwd` returns the file's contents directly.

IMPACT

- On a standard server deployment: disclosure of arbitrary files readable by the process, including `/etc/passwd`, source code, and credential files.
- Server-side request forgery against internal services via the `http://` entity scheme.
- Potential denial of service through entity-expansion techniques such as the 'billion laughs' attack.
- On older or Java-based XML stacks, this class of bug has historically led to full remote code execution.

ROOT CAUSE

`app.py` parses untrusted XML with the default `xml.sax` parser and never sets the `external-general-entities`, `external-parameter-entities`, or `external DTD-loading` features to `False`.

RECOMMENDED FIX

- ✓ Explicitly disable external entity resolution on the SAX parser via `setFeature()` for `external-general-entities`, `external-parameter-entities`, and `external DTD loading`.
- ✓ Preferably replace the standard library parser with `defusedxml`, which is purpose-built to reject DTDs and external entities by default: `from defusedxml import sax; sax.parseString(raw, handler)`.
- ✓ Never parse untrusted XML with the plain standard-library SAX or DOM parsers.

Verification after fix: The same payload returns an explicit 'external entity not allowed' error with no content expansion, while legitimate XML without a DOCTYPE continues to parse correctly.

Finding 14 — Weak / Hardcoded Session Secret Key

CRITICAL

CVSS: 9.8 Location: app.py — app.secret_key CWE: CWE-798, CWE-321 Status: Confirmed — directly exploited in Finding 5

DESCRIPTION

The Flask application's session-signing secret key is hardcoded in source as the string "insecure-secret-key-for-learning". Flask uses this key to cryptographically sign session cookies; because the key is fixed, guessable, and visible to anyone with access to the source (including via Findings 6, 7, 9 and 11), any attacker who obtains it can forge a validly signed session for any user, including an administrator, as demonstrated in Finding 5.

STEPS TO REPRODUCE

1. Obtain the source code by any of the routes already demonstrated in this report (path traversal, command injection, SSRF, or SSTI).
2. Locate `app.secret_key` in the source and note its fixed value.
3. Use Flask's own `SecureCookieSessionInterface` to sign an arbitrary session payload with that key.
4. Send the resulting cookie to any protected endpoint to gain unauthenticated access.

PROOF OF CONCEPT

Reused forged cookie (as shown in Finding 5)

```
eyJ1c2VyX2lkIjoyLCJyb2x1IjoiYWRTaW4ifQ.aryEMA.WpeKEUuP6HiiKoIzV4zDxcj5PTk
```

This cookie, signed offline using the known hardcoded key, was accepted by the `/admin` endpoint with no login having occurred, confirming the key is both known and actively trusted by the server.

IMPACT

- Enables impersonation of any user, including administrators, once the key is known.
- The same key signs any other data Flask signs on the application's behalf, so its compromise is not limited to session cookies alone.
- Combined with Finding 5 (broken access control), this single weakness is sufficient to fully compromise the application's authentication model.

ROOT CAUSE

`app.py` contains the line `app.secret_key = "insecure-secret-key-for-learning"`, committed directly to source and never rotated.

RECOMMENDED FIX

- ✓ Generate a strong, cryptographically random secret at deploy time: `app.secret_key = secrets.token_hex(32)`.
- ✓ Load the key from an environment variable or dedicated secrets manager, never from source code:
`app.secret_key = os.environ["SECRET_KEY"]`.
- ✓ Use a distinct key per environment (development, staging, production) — never reuse one across environments.
- ✓ Rotate the key immediately if it is ever suspected to have been exposed, and periodically as routine hygiene.
- ✓ Ensure secrets are excluded from version control via `.gitignore` and managed through a proper secrets manager (e.g. HashiCorp Vault, AWS Secrets Manager).

Verification after fix: After rotating to a strong, environment-sourced key, all previously forged cookies fail signature verification, new sessions are signed with the new key, and the key can no longer be predicted or derived from the source code.

Finding 15 — Cross-Site Request Forgery (CSRF) in Password Change

MEDIUM

CVSS: 6.5 Location: /password CWE: CWE-352 Status: Confirmed

DESCRIPTION

The /password endpoint accepts POST requests that change the current user's password, and its only check is that the user is logged in. It requires no CSRF token, performs no verification of the request's origin, and does not require re-entry of the current password. As a result, any third-party site can cause a logged-in victim's browser to silently submit a password-changing request on their behalf.

STEPS TO REPRODUCE

1. Log in to the application as felix in the browser.
2. Visit an attacker-controlled page containing an auto-submitting HTML form that POSTs to /password with a new password value.
3. Observe that the browser submits the form automatically, attaching the victim's existing session cookie.
4. Confirm the password was changed server-side with no confirmation step, then log in with the attacker-chosen password to prove takeover.

PROOF OF CONCEPT

Full CSRF chain reproduced with curl

```
Login to capture a session cookie, then POST to /password with a new value
and the saved cookie but no CSRF token
```

The server responded 'Password updated.' with no token or origin check performed. A follow-up login attempt using the new password succeeded, confirming the change took effect purely through the forged request.

IMPACT

- Silent password change of any logged-in user who can be induced to load an attacker's page.
- Full account takeover once the attacker subsequently logs in with the password they chose.
- Requires no interaction beyond the victim visiting a malicious page, or even loading an embedded image or link containing the hidden form.
- Becomes trivial to weaponise at scale if chained with the stored XSS in Finding 3 or the open redirect in Finding 10.

ROOT CAUSE

app.py's /password handler updates the database directly from the submitted 'new' field with no CSRF token validation and no requirement to supply the current password.

RECOMMENDED FIX

- ✓ Add CSRF protection via Flask-WTF (`CSRFProtect(app)`) and include the resulting token as a hidden field in every state-changing form.
- ✓ Require re-entry of the current password to authorise any change — this alone renders blind CSRF useless, since the attacker cannot know it.
- ✓ Set the session cookie's SameSite attribute to Lax or Strict, so it is not sent on cross-site requests.
- ✓ Check the Origin or Referer header on state-changing requests as an additional layer of defence.
- ✓ Use POST exclusively for state-changing operations, never GET.

Verification after fix: A POST without a valid CSRF token is rejected with HTTP 400, a cross-site page can no longer trigger a password change, and requiring the current password blocks any residual blind-CSRF attempts.

Finding 16 — Werkzeug Interactive Debugger Exposed (Debug Mode Enabled)

CRITICAL

CVSS: 9.8 **Location:** Any URL that triggers an unhandled exception **CWE:** CWE-94, CWE-489 **Status:** Confirmed

DESCRIPTION

The Flask application runs with `debug=True`. When an unhandled exception occurs, Werkzeug serves a full interactive debugger page containing a complete traceback with file paths and line numbers, source code excerpts from the application, and — critically — an interactive Python console capable of executing arbitrary code in the context of the failed request. An attacker only needs to trigger any unhandled error to obtain this level of access.

STEPS TO REPRODUCE

1. Trigger an unhandled exception with malformed input, e.g. `/profile?id=abc`.
2. Observe that the server returns a full Werkzeug debugger page rather than a generic error.
3. Review the disclosed traceback, which includes exact source file paths, line numbers, and a message stating that arbitrary Python code can be executed in the stack frames shown.

PROOF OF CONCEPT

Triggered traceback disclosure

```
/profile?id=abc
```

The response was a full Werkzeug debugger page showing a `sqlite3.OperationalError` with a complete traceback, including the exact vulnerable line of source code, and Werkzeug's own message confirming that arbitrary Python code execution is available through the console. On a desktop browser, clicking any frame and then the console icon opens a live Python prompt inside the running server process; this icon did not render in the mobile browser used for testing, but the vulnerability is fully confirmed by the disclosure itself, since Werkzeug's debugger provides remote code execution by design whenever it is reachable.

IMPACT

- Complete disclosure of application source code, file paths, and internal logic on any error.
- Direct disclosure of the Flask secret key and any other in-scope secrets referenced in the traceback, feeding directly into Finding 5.
- Arbitrary Python code execution within the server process wherever the interactive console can be reached.
- Considered one of the most severe possible misconfigurations, since it converts any application bug into a full server compromise.

ROOT CAUSE

`app.py` starts the server with `app.run(host="127.0.0.1", port=5000, debug=True)`. Debug mode is never safe outside of local development on a trusted machine.

RECOMMENDED FIX

- ✓ Never run with `debug=True` outside local development: `app.run(debug=False)`.
- ✓ Control debug mode through an environment variable that defaults to off: `debug = os.environ.get("FLASK_DEBUG", "0") == "1"`.
- ✓ Register explicit error handlers that return a generic message with no traceback: `@app.errorhandler(Exception)` returning a plain 'An error occurred' response.
- ✓ Log exceptions to a server-side log file only, never to the browser.
- ✓ Fix the underlying cause of the exception (the unparameterised query in the `/profile` route) so malformed input does not trigger an error in the first place.

Verification after fix: `/profile?id=abc` returns a generic HTTP 500 page with no traceback, no source disclosure, no console, and no line numbers.

6. Consolidated Recommendations

While each finding above includes a specific fix, the following cross-cutting recommendations address the root patterns behind most of the issues identified and should be prioritised as a programme rather than fixed one endpoint at a time:

- 1 Treat all user input as untrusted data, never as code:** use parameterised SQL queries everywhere, pass variables into templates rather than building templates from strings, and never call `shell=True` or `pickle.loads` on anything the client can influence.
- 2 Enforce output encoding by default:** render all dynamic content through an auto-escaping template engine (Jinja's `{{ }}` syntax) rather than raw f-string HTML construction, and never apply the `|safe` filter to user-controlled data.
- 3 Verify authorisation on the server for every request:** derive identity and role from the server-side session and database on each request, never from a client-supplied id or a claimed role field.
- 4 Replace the hardcoded secret key** with a strong, environment-sourced value, and rotate it immediately given how many findings in this report already depend on its disclosure.
- 5 Disable Flask's debug mode** in any environment beyond a developer's own machine, and replace it with generic error pages backed by server-side logging.
- 6 Apply baseline HTTP security headers globally** (Content-Security-Policy, X-Content-Type-Options: nosniff, and cookie flags `HttpOnly / Secure / SameSite`) via a single `after_request` hook rather than per-route.
- 7 Introduce CSRF protection** (Flask-WTF) and re-authentication for all state-changing actions such as password changes.
- 8 Restrict file uploads** to an extension allow-list, sanitise filenames, store uploads outside the web root, and serve them as forced downloads rather than inline content.
- 9 Constrain any endpoint that fetches URLs or reads files** on the server's behalf (SSRF/LFI/XXE surfaces) to an explicit allow-list of schemes, hosts, or directories.
- 10 Adopt a secure software development lifecycle practice** such as a pre-release checklist against the OWASP Top 10, and consider periodic automated dependency and static-analysis scanning going forward.

7. Ethical and Legal Considerations

All testing described in this report was conducted against VulnLab, a training application deliberately built by the tester for this project, running exclusively on a local, isolated host under the tester's own control. No production system, third-party service, or another person's data was accessed at any point, and all payloads used were the minimum necessary to confirm each vulnerability rather than to cause damage.

This distinction is the foundation of ethical hacking as a discipline: identical techniques performed without explicit, informed authorisation from the system owner constitute unauthorised computer access and are unlawful in most jurisdictions (for example under the UK Computer Misuse Act 1990 or the US Computer Fraud and Abuse Act). A responsible tester therefore always operates strictly within an agreed scope, obtains written authorisation before testing, avoids any action that could cause denial of service or data loss beyond what is necessary to demonstrate a finding, and reports findings responsibly and confidentially to the system owner rather than disclosing them publicly.

In line with this principle, the findings in this report are documented solely for educational and portfolio purposes, demonstrating a structured, methodology-driven approach to vulnerability identification and remediation on a self-built lab.

8. Conclusion

This assessment found that VulnLab contains a broad and representative cross-section of the OWASP Top 10 vulnerability classes, including injection (SQL, command, template, and XML), broken authentication and session management, broken access control, insecure deserialization, security misconfiguration, and insufficient input validation. Multiple findings independently allow full remote code execution, meaning the application's confidentiality, integrity, and availability are all at risk in its current state.

The exercise demonstrates, in a safe and contained environment, how individually modest oversights — a missing escape call, a trusted client-side role claim, a debug flag left on — compound into complete system compromise when present together. Applying the fixes detailed in Section 5, guided by the cross-cutting priorities in Section 6, would remediate every finding identified and bring the application in line with standard secure-coding practice for a Flask web application.

Report prepared by: Felix Otieno | **GitHub:** github.com/felixotieno1011-art | **Date:** 30 September 2026

This report was produced as part of an ethical hacking portfolio project on a self-built, deliberately vulnerable lab application.